Listing 13.6 shows a possible implementation of this method, that treats argument `clustersSet` like an instance of disjoint set shared among all reducers. While this is not practically possible[20], it is conceptually equivalent to having the reducers emitting a list of clusters to merge, and perform the operations on the disjoint set in the combiner stage, after reducers finish their job. For this reason, we can consider `clusterSet` like a facade[21] simplifying the process of emitting a pair of clusters to merge and sending the pair to the combiner, where the actual disjoint set is created and merge operations are performed.

Listing 13.6 method mergeClusters

```
function mergeClusters(shard1, shard2, marginPoints,
clustersSet)              #1
  for p in marginPoints.intersection(shard1, shard2)
do                        #2
    if shard1.isCorePoint(p) or shard2.isCorePoint(p)
then                     #3
      if shard1.isNoise(p)
then                                                  #4
        shard1.markNoise(p,
false)                                               #5
      elsif shard2.isNoise(p)
then                                                #6
        shard2.markNoise(p, false)
      else
        clustersSet.merge(shard1.getCluster(p),
shard2.getCluster(p)) #7
  return clustersSet, shard1, shard2
                                  #8
```